

SHUNYA Constitutional Architecture — Version 1.0

Incorporates: LX-06A (Runtime Hierarchy + Canonical Ownership) • LX-06B (Experience • Objects • Cognition • Navigation • Runtime Contracts) • Final Directive (Object Lifecycle • Ownership Law • Expanded Contracts • Layer Hierarchy • Object Interaction • Event Law • Workspace Purity • Simplicity Rule • Experience Metrics • Freeze)

Status: Frozen. All future architectural evolution shall occur exclusively through the Constitutional Amendment Process. No implementation may knowingly violate a constitutional invariant, even temporarily, without an explicit constitutional amendment.

Preamble

SHUNYA is not a collection of runtimes serving features.

SHUNYA is a **constitutional operating system** where every component exists to serve **Living Objects** within a **continuous founder experience**.

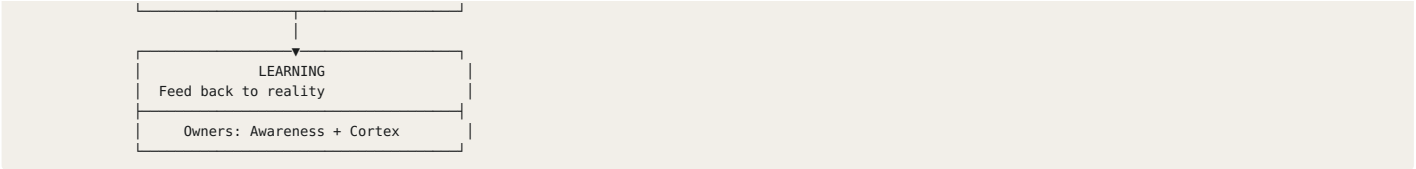
No runtime is the primary abstraction. Objects are. No feature exists without improving experience. Every layer has contracts it may not violate.

Part I — The Constitutional Stack

1. The Constitutional Hierarchy

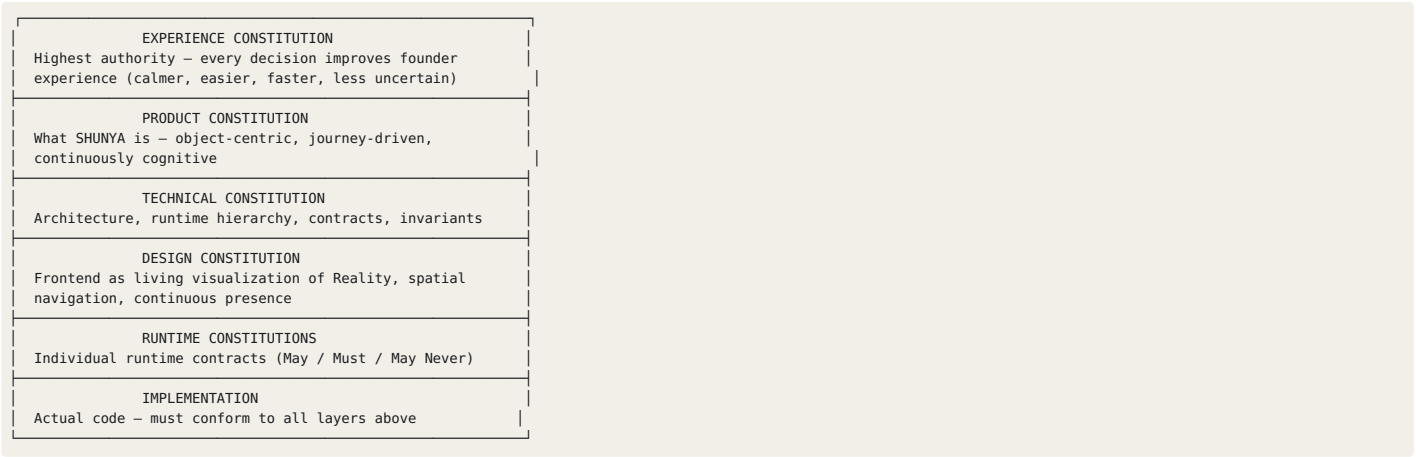
Experience is the highest constitution. Reality is not the first layer — Experience is.





2. Constitutional Layer Hierarchy

Constitutional precedence is frozen as follows. Lower layers may never contradict higher layers.



3. Experience Constitution

Every capability must answer:

Question	What it measures
What does the founder now understand immediately?	Clarity
What uncertainty disappeared?	Certainty
What cognitive effort disappeared?	Cognitive load
What work disappeared?	Manual effort
What became continuous?	Continuity
What became calmer?	Calm

No runtime may exist without improving at least one of these.

Experience is **not a runtime**. Experience is the constitutional purpose for every runtime.

4. Constitutional Simplicity Rule

Whenever two constitutional solutions satisfy the same Experience objective, the simpler architecture shall prevail.

Additional abstraction requires constitutional justification.

Complexity is not a feature. Complexity that does not serve Experience is a defect.

Part II — Living Objects Constitution

5. Objects as Primary Abstraction

SHUNYA is **object-centric**, not runtime-centric.

Runtimes exist only to serve Living Objects. No runtime shall become the primary abstraction of SHUNYA.

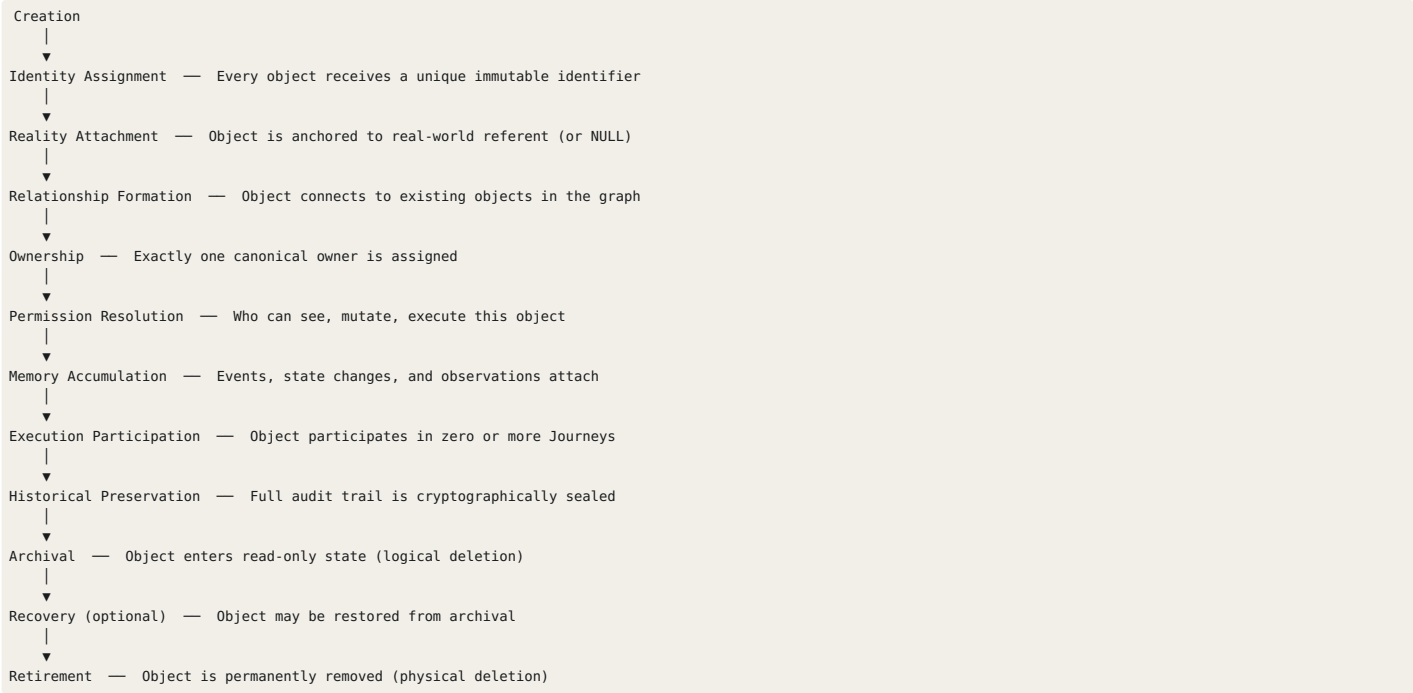
6. Canonical Living Objects

Object	Description	Backend Owner	Frontend Owner
Identity	A person — immutable sid_xxx	identity_repository.py	session/auth
Organization	A group of identities	models.py Org	workspace store
Workspace	Reality relevant to a human	objects/models.py	workspace-runtime
Conversation	A business dialogue	conversation-runtime (FE)	conversation frontend
Customer	A business relationship	relationships/	relationship graph
Relationship	Connection between objects	relationship/	object-graph-runtime
Proposal	A business offer	FOR-1/2	proposal components
Commitment	An agreed outcome	commitment-runtime (FE)	commitment frontend
Execution	Work in progress	execution/	timeline frontend
Invoice	A payment request	finance/	finance components
Payment	A financial transaction	razorpay/	finance components
Journey	A founder experience arc	Journey composition	journey components
Document	A persisted artifact	upload/ + cloudinary/	document components
Knowledge	A stored fact	knowledge_store.py	intelligence runtime
Memory	Past observations	state-fabric + memory	state fabric
Decision	A chosen course	decision_runtime/	commitment runtime
Outcome	A completed result	outcome_engine.py	experience engine
Observation	An event noticed	events/	realtime svnc

Creation	API Event Hub	State	Realtime Sync
----------	---------------	-------	---------------

7. Universal Object Lifecycle

Every canonical object shall implement the same lifecycle. No object type may invent an alternative lifecycle unless the Constitution explicitly extends this one.



Lifecycle invariants: - Every object must reach at minimum **Identity Assignment** before being visible - **Memory Accumulation** begins at Creation and never stops until Retirement - **Archival** is reversible (Recovery). **Retirement** is not. - No lifecycle stage may be skipped without a constitutional extension

8. Object Contract

Every Living Object must:

1. Have a unique canonical identity
2. Be addressable via the Universal Object API (`/api/v1/objects/{id}`)
3. Publish lifecycle events (created, updated, archived, retired)
4. Be traversable via the Object Graph
5. Have an owner (backend + frontend)
6. Participate in at least one Journey
7. Expose its current lifecycle stage

9. Object Interaction Law

Whenever one Living Object affects another, the Constitution requires:

Requirement	Description
Causality	Which object initiated the interaction and why
Evidence	Proof the interaction occurred (event, log, side effect)
Timeline	When the interaction began, progressed, and completed
Rollback History	Every mutation is reversible to a known prior state
Explanation	A human-readable reason the interaction was necessary

No invisible mutation may occur. Every object-to-object interaction must leave an auditable trace.

Part III — Canonical Ownership

10. Canonical Ownership Law

Every architectural element shall have **exactly one canonical owner**.

This includes:

runtime	API	event	state
object	workspace	frontend	component
command	cognition	stream	memory
journey	layout	provider	transport
service	blueprint	database table	integration

If ownership is ambiguous, the implementation is **constitutionally invalid**.

11. Ownership Assignments

Concern	Canonical Owner	Backend	Frontend
Reality	RealityEngine	<code>reality_engine/engine.py</code>	living-store (consumer)
Observation	Delta Events	<code>events/</code>	use-realtime-sync
Memory (State)	State Fabric	—	state-fabric
Memory (Knowledge)	Knowledge Store	<code>knowledge_store.py</code>	—
Memory (Identity)	Identity Repository	<code>identity_repository.py</code>	—
Memory (Auth)	Authz Engine	<code>authz/</code>	—
Attention	Orchestrator	—	orchestrator.ts

Understanding	Intelligence Engine	<code>intelligence/</code>	<code>intelligence-runtime</code>
Cognition	Continuous Cognition	<code>intelligence/</code> (canonical publisher)	<code>ai-presence-panel</code>
Prediction	Prediction Engine	<code>prediction/engine.py</code>	—
Decision	Decision Runtime	<code>decision_runtime/</code>	<code>commitment-runtime</code>
Execution	Execution Runtime	<code>execution/</code>	<code>object-runtime</code>
Outcome	Outcome Engine	<code>outcome_engine.py</code>	<code>experience-engine</code>
Learning	Awareness + Cortex	<code>awareness/ + cortex/</code>	—
Workspace	Workspace Runtime	<code>objects/models.py</code>	<code>workspace-runtime</code>
Object	Object Runtime	<code>objects/</code>	<code>object-runtime</code>
Journey	Journey Composition	(composed of objects)	<code>experience-engine</code>
Command Surface	Living Command Surface	—	<code>living-workspace/command-surface.tsx</code>
AI Presence	AI Presence	<code>ai/</code>	<code>ai-presence-panel.tsx</code>
Relationship	Relationship Graph	<code>relationship/</code>	<code>object-graph-runtime</code>
Notification	Notification Context	<code>integration/models.py</code>	<code>NotificationContext</code>
Search	Search Engine	<code>search/</code>	—
API Client	Client	—	<code>api/client.ts</code>
Component Registry	Component Registry	—	<code>lib/component-registry.ts</code>
Event Bus	Runtime Event Bus	—	<code>runtimes/event-bus.ts</code>
Design Tokens	Token Provider	—	<code>tokens/</code>
Layout	Layout Engine	—	<code>layout-engine</code>

Part IV — Runtime Contracts

12. Constitutional Runtime Contract Specification

Every runtime shall constitutionally declare:

Field	Description
Canonical Owner	The team/person/blueprint responsible for this runtime
Purpose	Why this runtime exists — which Experience question it answers
Reality Inputs	What real-world events this runtime consumes
Published Outputs	What this runtime produces for other runtimes
Events Consumed	Event types this runtime subscribes to
Events Published	Event types this runtime emits
State Owned	State this runtime permanently owns
State Forbidden	State this runtime may never own
Dependencies	Other runtimes this runtime depends on
Extension Points	Where this runtime can be extended without violating contracts
Failure Behaviour	What happens when this runtime fails
Recovery Behaviour	How this runtime returns to normal operation
Forbidden Responsibilities	What this runtime may never do

No runtime may own undeclared state.

13. Runtime Contracts

Runtime	Purpose	May	Must	May Never	State Owned	State Forbidden	Failure Behaviour
Reality Engine	Compose what exists	Compose events, publish state, coordinate engines	Be the single source of truth	Render UI, mutate business objects directly	Engine registry, event composition graph	UI state, workspace state	Degrade gracefully — continue serving cached reality
Delta Events	Deliver observations	Poll, stream, publish observations	Deliver events to subscribers	Mutate memory directly	Subscription registry, delivery cursor	Persistent object state	Queue events for replay on recovery
State Fabric	Versioned state persistence	Persist, snapshot, restore state	Provide versioned, auditable state	Make decisions	State snapshots, persistence config	Decision history, execution plans	Snapshots survive — restore from last valid
Knowledge Store	Fact storage and retrieval	Store, retrieve, search facts	Keep knowledge queryable	Execute workflows	Knowledge base index	Execution state, workspace context	Serve cached knowledge, reconnect on restore
Identity Repository	Identity lifecycle	Create, read, update identities	Keep identity kernel-compliant	Bypass authz gates	Identity records, auth method mappings	Business object state	Read-only mode — no new identities until recovery
Orchestrator	Runtime lifecycle and health	Register runtimes, resolve DAG, monitor health	Manage runtime lifecycle	Execute business logic	Runtime registry, health probes	Object data	Restart failed runtimes recursively
Workspace Runtime	Context for the current human	Provide context, focus, continuity, memory restoration	Render Reality for the current human	Own business logic	Active workspace, user context, navigation history	Business rules, AI reasoning	Restore last known good workspace
Intelligence Engine	Pattern recognition	Infer, reason, explain, detect patterns	Provide explainable insights	Execute decisions	Inference cache, relationship index	Execution commands	Degrade — serve cached insights
Continuous Cognition	Expose SHUNYA's current thinking	Publish observations, reasoning, confidence	Continuously expose what SHUNYA is thinking	Execute decisions	Cognition state (current thoughts)	Decision commands, execution state	Pause cognition, resume with context
Object Graph	Relationship traversal	Link, unlink, traverse objects	Maintain relationship integrity	Mutate objects	Graph edge index	Object payloads	Rebuild from object events
Prediction Engine	Simulation and foresight	Simulate, forecast, scenario analysis	Estimate likelihood and impact	Commit resources	Scenario models, prediction cache	Execution commands	Invalidate predictions on state change
Decision Runtime	Policy and commitment	Recommend, commit, enforce policies	Record decisions with evidence	Execute workflows	Decision log, policy registry	Object payloads	Block decisions, require human override
Execution Runtime	Outcome completion	Execute outcomes, schedule work, manage recovery	Complete outcomes with verification	Decide what to execute	Execution queue, outcome registry	Decision records, AI reasoning	Retry with backoff, escalate after N failures

Outcome Engine	Verification and certification	Verify completion, collect evidence	Certify outcomes or flag failures	Circumvent reality	Outcome registry, evidence store	Execution commands	Flag incomplete as failed
Awareness + Cortex	Organizational learning	Learn, adapt, update organizational memory	Feed learning back into Reality	Observe same event twice identically	Learning state, organizational patterns	Business objects	Stale learning — resume fresh
AI Provider	LLM inference routing	Route LLM requests, fallback chain	Provide reliable AI inference	Own business logic	Provider registry, failure counts	User data (pass through only)	Fallback through chain
Command Surface	Intent capture	Accept intent, route to execution	Surface available actions	Interpret intent independently	Command history	Business logic	Degrade to basic commands

14. Universal Event Law

Every published event must identify:

Field	Required	Description
source	Yes	Which runtime or component emitted the event
owner	Yes	Canonical owner of the event stream
affected_object	Yes	Which Living Object (by ID) this event concerns
timestamp	Yes	When the event occurred (UTC ISO 8601)
causality	Yes	What chain of events led to this event
confidence	Yes	How certain the system is about this event's correctness
reversibility	Yes	Whether this event can be rolled back

Anonymous events are constitutionally forbidden.

Part V — Navigation & Workspace Constitution

15. Navigation Constitution

SHUNYA shall never be page-centric.

Principle	Implementation
Movement must preserve context	No hard <code>window.location.href</code> navigations
History must preserve cognition	Navigation history includes cognition state
Objects must remain continuous	Object state survives workspace transitions
Transitions must feel spatial	Spatial transitions, not page loads
Founder never mentally restarts	Workspace restores exact prior context

16. Workspace Constitution (Purity Law)

Workspace is not a screen. Workspace is not navigation.

Workspace is **the continuously evolving manifestation of Reality relevant to the current human**.

Workspace **shall**: - Visualize Reality - Preserve Context - Preserve Continuity - Preserve Presence

Workspace **shall never own**: - Business rules - Persistence logic - Workflow logic - AI reasoning - Domain computation

Workspace remains a **projection layer only**. It renders Reality; it does not compute Reality.

17. Frontend Constitution

The frontend is not a presentation layer.

It is the **living visualization of Reality**.

Every component must answer:

Question	What it reveals
What Reality does this visualize?	Source of truth
What Object does this represent?	Living Object identity
What AI cognition does it expose?	Current thinking
What execution does it enable?	Actionability
What next action does it surface?	Next step

Components that merely display data violate the constitution.

Part VI — Journey Constitution

18. Founder Journeys

Founder Journeys become **first-class architectural primitives**.

Every journey shall explicitly define:

Stage	Description	Example (Customer → Revenue)
Reality	What exists	A lead appears
Understanding	What it means	Lead scored, needs assessed
Decision	What to do	Create proposal
Execution	Make it happen	Send proposal, negotiate
Outcome	What resulted	Signed or lost
Learning	Feed back	Win/loss analysis

Canonical Journeys (minimum):

- Identity Onboarding** — Person becomes Identity → creates Organization → enters Workspace

2. **Customer Acquisition** — Lead → Conversation → Proposal → Negotiation → Approval → Invoice → Payment → Delivery → Feedback → Future Opportunity
3. **Commitment Management** — Intent → Commitment → Execution → Evidence → Outcome → Learning
4. **Knowledge Discovery** — Question → Search → Understanding → Synthesis → Memory → Application

Part VII — Canonical Experience Pipeline

19. Single Execution Path

Every interaction shall permanently follow this pipeline:

Reality → Objects → Attention → Understanding → Cognition → Decision → Execution → Outcome → Learning → Updated Reality

No alternative execution path shall exist.

If a component cannot trace its execution path through this pipeline, it violates the constitution.

Part VIII — Blueprint & File Convergence

20. Blueprint Ownership (Backend)

(As previously established — 34 blueprints, ~25 after convergence.)

21. Eliminated Files

(As previously established — ~22 files, 12 frontend + 10 backend.)

Part IX — Experience Metrics

22. Convergence Measurement

Architectural success is no longer measured only through engineering metrics.

Every convergence shall demonstrate measurable reduction in:

Metric	Description	Target
Clicks removed	Interaction count before/after	Fewer
Cognitive effort removed	Steps to complete a task before/after	Less
Uncertainty removed	Unknown or ambiguous states before/after	Less
Waiting removed	Time spent waiting for the system before/after	Less
Navigation removed	Transitions required before/after	Fewer
Duplicated understanding removed	Times founder must re-learn before/after	Zero
Architectural complexity removed	Files, blueprints, runtimes before/after	Less
Continuous visibility	Polling → streaming before/after	Always
Journey completion speed	End-to-end time per journey before/after	Faster
Founder confidence	Ability to act before/after	Higher

Experience improvement shall be **objectively measurable**.

Part X — Acceptance Gates

23. Final Convergence Conditions

LX-06 is complete only when:

#	Gate	Evidence
1	✓ Experience is the constitutional top layer	This document, Section 1
2	✓ Living Objects are the primary architectural abstraction	Sections 5-9, object API
3	✓ Universal Object Lifecycle is defined	Section 7
4	✓ Every architectural element has exactly one owner	Sections 10-11
5	✓ Continuous Cognition has one canonical owner	app/intelligence/ as publisher
6	✓ Workspace is a projection layer only	Section 16 (Workspace Purity Law)
7	✓ Navigation follows the continuity constitution	Section 15
8	✓ Founder Journeys are architectural primitives	Section 18
9	✓ Every runtime has explicit expanded contracts	Sections 12-13
10	✓ Constitutional layer hierarchy is frozen	Section 2
11	✓ Object Interaction Law is defined	Section 9
12	✓ Universal Event Law is defined	Section 14
13	✓ Constitutional Simplicity Rule is adopted	Section 4
14	✓ Experience metrics accompany engineering metrics	Section 22
15	✓ Frontend is defined as living visualization of Reality	Section 17
16	✓ Every convergence maps to this constitutional architecture	Convergence plan
17	✓ Every constitutional runtime has one owner	Hierarchy in Section 1
18	✓ Every architectural concern has one owner	Ownership in Section 11
19	✓ Every duplicated implementation merged or delegated	File elimination plan
20	✓ Every convergence includes dependency evidence	Dependency graph template
21	✓ Backend and frontend share the same architecture	This document maps both
22	✓ Founder experience has measurably improved	Experience metrics per convergence
23	✓ Complexity has decreased	Files, blueprints, runtimes reduced

24	✓ No regressions introduced	Full test suite + founder walkthrough
----	-----------------------------	---------------------------------------

Part XI — Constitutional Amendment Process

24. How the Constitution Changes

1. A constitutional amendment must identify which article it changes
2. It must explain why the current article no longer serves Experience
3. It must specify how the amendment improves: Understanding, Execution, Trust, or Adaptation
4. It must be approved by the founder
5. After approval, the constitution is re-frozen until the next amendment

25. Constitutional Freeze

Effective immediately:

SHUNYA Constitutional Architecture V1.0 is frozen.

- All future architectural evolution shall occur exclusively through the Constitutional Amendment Process
- No implementation may knowingly violate a constitutional invariant, even temporarily, without an explicit constitutional amendment
- The Constitution is the permanent architectural standard for SHUNYA

END OF CONSTITUTION — VERSION 1.0

This document is the frozen V1.0 architectural constitution for SHUNYA. All convergence, migration, deletion, and new feature development shall conform to this constitution. No future implementation may violate these constitutional principles unless the constitution itself is explicitly amended.